

Reviewer Pack — Minimal Rank of Hodge Classes over Tseitin Circuit Width

Entry af48c5f5dddf · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 13:30:12 UTC

Minimal Rank of Hodge Classes over Tseitin Circuit Width

Entry ID: af48c5f5dddf **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 13:30:12 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Theory) **Field B** (complexity object): Complexity Theory: Tseitin Circuit Width

Statement:

[‘For every Tseitin formula F on n variables, the minimal rank of a Hodge class of degree equal to the width of F is $\Theta(n^2)$.’, ‘Equivalently, for all n and F with Tseitin circuit width $w(F)$, there exists a Hodge class H such that $\dim(H) = O(w(F)^2)$.’, ‘If F is a Tseitin formula on n variables with width $w(F)$, then the minimal rank of any Hodge class associated with F is at least $\Omega(n^2 / w(F)^2)$.’]

Rationale (proposer’s reasoning):

[‘Hodge theory, which studies algebraic varieties and their cohomology, has not been extensively applied to complexity theory. This conjecture proposes a connection between the geometric property of Hodge classes and the complexity of Tseitin circuits, which could expose new structural insights.’, ‘Tseitin circuit width is an important measure of circuit complexity that can be used to distinguish between NP-hard problems. By relating this measure to Hodge theory, we might find a new way to analyze and potentially solve NP-hard problems.’, ‘This conjecture builds on the idea that geometric properties of certain algebraic structures could correspond to computational complexities, which is an area with significant potential for new discoveries.’]

Taxonomy category: TSEITIN_RES (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c6ea109996f195c6

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For each Tseitin formula F on n variables with varying widths $w(F)$, if the minimum observed rank of Hodge classes is $\geq O(n^2 / w(F)^2)$ for all seeds and the mean rank across seeds is $\leq 1.5 * O(n^2 / w(F)^2)$, then support for the conjecture is provided.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - algebraic geometry AND hodge theory AND tseitin circuit width - hodge classes AND tseitin formula AND complexity theory - minimal rank AND Hodge Theory AND Tseitin circuit width

Top relevant hits considered: - [http://arxiv.org/abs/2102.03481v3] Noncommutative Hodge conjecture - [http://arxiv.org/abs/0707.3937v1] Cohomology theories for homotopy algebras and noncommutative geometry - [http://arxiv.org/abs/1602.06607v4] Why should one compute periods of algebraic cycles? - [http://arxiv.org/abs/1008.4299v3] Characteristic classes of symmetric products of complex quasi-projective varieties - [http://arxiv.org/abs/1801.10489v3] Hodge level for weighted complete intersections - [http://arxiv.org/abs/2103.09609v1] Characterizing Tseitin-formulas with short regular resolution refutations - [http://arxiv.org/abs/1803.01936v4] Variations of Hodge Structures of Rank Three k-Higgs Bundles and Moduli Spaces of Holomorphic Triples - [http://arxiv.org/abs/1403.6443v3] The Hodge-de~Rham Theory of Modular Groups

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def tseitin_formula(n):
        variables = [f'x{i}' for i in range(1, n+1)]
        clauses = []
        for var in variables:
            clauses.append(f'{var} | ~{var}')
        return ' & '.join(clauses)

    def hodge_class_rank(n):
        # Placeholder function to simulate Hodge class rank computation
        # This is a dummy implementation and should be replaced with actual logic
        return n * (n + 1) // 2

    def tseitin_circuit_width(formula):
        # Placeholder function to simulate Tseitin circuit width computation
```

```

# This is a dummy implementation and should be replaced with actual logic
return len(formula.split(' & '))

results = []
for n in [5, 10, 15, 20, 30, 40]:
    formula = tseitin_formula(n)
    width = tseitin_circuit_width(formula)
    rank = hodge_class_rank(n)
    results.append({
        "metric_name": "hodge_class_rank",
        "metric_value": rank,
        "instances_tested": 1,
        "conjecture_holds": rank >= n**2 / width**2 and rank <= 1.5 * n**2 / width**2,
        "counterexample": ""
    })

mean_rank = sum(r["metric_value"] for r in results) / len(results)
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

return {
    "seed": seed,
    "mean_rank": mean_rank,
    "support_fraction": support_fraction
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [389, 421, 463, 503, 547, 593, 631, 677, 727, 773, 811]
    results = [run_trial(seed) for seed in seeds]

    print("TRIALS:")
    for result in results:
        print(f"TRIAL: {result}")

    mean_rank = sum(result["mean_rank"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["support_fraction"] >= 0.8) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std=NA support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=NA first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

'support_fraction': 0.0}
TRIAL: {'seed': 211, 'mean_rank': 280.8333333333333, 'support_fraction': 0.0}
TRIAL: {'seed': 233, 'mean_rank': 280.8333333333333, 'support_fraction': 0.0}
TRIAL: {'seed': 257, 'mean_rank': 280.8333333333333, 'support_fraction': 0.0}
TRIAL: {'seed': 277, 'mean_rank': 280.8333333333333, 'support_fraction': 0.0}
TRIAL: {'seed': 311, 'mean_rank': 280.8333333333333, 'support_fraction': 0.0}

```

```

TRIAL: {'seed': 347, 'mean_rank': 280.8333333333333, 'support_fraction': 0.0}
TRIAL: {'seed': 389, 'mean_rank': 280.8333333333333, 'support_fraction': 0.0}
TRIAL: {'seed': 421, 'mean_rank': 280.8333333333333, 'support_fraction': 0.0}
TRIAL: {'seed': 463, 'mean_rank': 280.8333333333333, 'support_fraction': 0.0}
TRIAL: {'seed': 503, 'mean_rank': 280.8333333333333, 'support_fraction': 0.0}
TRIAL: {'seed': 547, 'mean_rank': 280.8333333333333, 'support_fraction': 0.0}
TRIAL: {'seed': 593, 'mean_rank': 280.8333333333333, 'support_fraction': 0.0}
TRIAL: {'seed': 631, 'mean_rank': 280.8333333333333, 'support_fraction': 0.0}
TRIAL: {'seed': 677, 'mean_rank': 280.8333333333333, 'support_fraction': 0.0}
TRIAL: {'seed': 727, 'mean_rank': 280.8333333333333, 'support_fraction': 0.0}
TRIAL: {'seed': 773, 'mean_rank': 280.8333333333333, 'support_fraction': 0.0}
TRIAL: {'seed': 821, 'mean_rank': 280.8333333333333, 'support_fraction': 0.0}
TRIAL: {'seed': 877, 'mean_rank': 280.8333333333333, 'support_fraction': 0.0}
TRIAL: {'seed': 929, 'mean_rank': 280.8333333333333, 'support_fraction': 0.0}

```

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3c9e3a11.py", line 71, in <module>
    if all(result["conjecture_holds"] for result in results):
    ~~~~~^~~~~~
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3c9e3a11.py", line 71, in <genexpr>
    if all(result["conjecture_holds"] for result in results):
    ~~~~~^~~~~~

```

KeyError: 'conjecture_holds'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating whether the conjecture holds or not. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12570
2	propose	ollama_remote	glm4:latest	0	0	11594
3	preregistration	ollama_remote	glm4:latest	0	0	5996
4	novelty	ollama_remote	glm4:latest	0	0	4702
5	novelty	ollama_remote	glm4:latest	0	0	6821
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15420
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7502
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7910
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8486

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
10	judge	ollama_remote	glm4:latest	0	0	9465

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 90466 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/af48c5f5dddf.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/af48c5f5dddf.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/af48c5f5dddf.tar.gz (if generated)